Examen de ejemplo: enunciados y soluciones

Examen práctico de Git, GitHub Pages y GitHub Actions (1 h 15 min, 10 puntos). Cada ejercicio lleva el **enunciado** (en castellano, con los nombres y mensajes en valenciano tal cual) y debajo su **solución**. Sustituye `<usuari>` por tu usuario de GitHub y `Nom Cognom` por tu nombre y primer apellido.

Para practicar: lee el enunciado, hazlo tú sin mirar la solución, y compara después.

Normas del examen

- Solo hay acceso a Aules y a github.com con su documentación. Buscadores y asistentes de IA están prohibidos.
- Ayuda local permitida: `git help <orden>`, `man git-<orden>` y `git <orden> -h`.
- Se corrige con el historial de los repos y la pestaña Actions de GitHub.
- Hay que usar exactamente los nombres de fichero, de repositorio y los mensajes de commit del enunciado.
- Si un ejercicio se bloquea, pasar al siguiente.
- `<usuari>` = usuario de GitHub. `<cognom>` = primer apellido, en minúsculas y sin acentos.
- Todos los repositorios deben ser públicos y crearse en la rama `main`.

Antes de empezar (no puntúa)

Enunciado

Configurar `user.name` y `user.email` de Git con tu nombre y el correo de GVA:

```
git config --global user.name "Nombre Apellido"
git config --global user.email "correo@ejemplo.com"
```

Configurar la autenticación en GitHub: crear un token con todos los permisos marcados, guardarlo en un fichero `auth.txt` y ejecutar:

```
gh auth login --with-token < auth.txt
gh auth setup-git
```

Solución

1. GitHub → foto de perfil → **Settings** → **Developer settings** → **Personal access tokens** → **Tokens (classic)** → **Generate new token (classic)**.

2. Nombre (por ejemplo `examen`), marcar **todos** los permisos (las casillas principales marcan también las secundarias) y **Generate token**. Copiar el token: solo se ve una vez.
3. Guardarlo en `~/auth.txt`, **fuera de los repositorios** (si lo subes a un repo, GitHub bloquea el push y revoca el token).

```
git config --global user.name "Nom Cognom"
git config --global user.email "correo@ejemplo.com"

cd ~
echo "PEGA_AQUI_EL_TOKEN" > auth.txt
gh auth login --with-token < auth.txt
gh auth setup-git
gh auth status          # comprobar
```

Ejercicio 1: Repo (2 puntos)

Enunciado

1. En vuestro GitHub, cread un repositorio llamado `exercici-1`.
2. En la VM, cread el directorio `~/examen` y entrad en él. Inicializadlo como repositorio y configuradlo para que el remoto sea el que habéis creado en el paso anterior.
3. Dentro del repositorio, cread el fichero `respostes.md` con esta línea: `# Respostes — <Nom Cognom>`
4. Haced un commit con el siguiente mensaje: `Primer commit`
5. Haced push al repositorio.
6. Cread un fichero llamado `Readme.md` y añadid vuestro nombre.
7. Haced un commit con el siguiente mensaje: `Segon commit`
8. Cread un fichero llamado `log.txt` y pegad en él el contenido de la salida de `git log --oneline`.
9. Haced un commit con el siguiente mensaje: `Tercer commit`

Solución

```
# 1. Repo vacío en GitHub (sin README, porque se sube desde local)
cd ~
gh repo create exercici-1 --public
# (o web: + → New repository → exercici-1 → Public → sin README)

# 2. Directorio local, inicializar y enlazar
```

```
mkdir ~/examen
cd ~/examen
git init -b main
git remote add origin https://github.com/<usuari>/exercici-1.git

# 3-5. Primer commit y push
echo "# Respostes — Nom Cognom" > respostes.md
git add respostes.md
git commit -m "Primer commit"
git push -u origin main

# 6-7. Segundo commit
echo "Nom Cognom" > Readme.md
git add Readme.md
git commit -m "Segon commit"

# 8-9. Tercer commit
git log --oneline > log.txt
git add log.txt
git commit -m "Tercer commit"

# Subir todo para que quede en GitHub
git push
```

- `git init -b main` crea el repo ya con la rama `main` (si no, `git branch -M main`).
- Copia el guion largo `—` de la línea de `respostes.md`.
- `log.txt` se crea antes de su propio commit: tendrá 2 líneas, y es lo esperado.
- Comprobación: `git log --oneline` debe mostrar `Tercer commit`, `Segon commit` y `Primer commit`; `git remote -v` debe apuntar a tu `exercici-1`.

Ejercicio 2: Repo (2 puntos)

Enunciado

1. Cread un repo en GitHub llamado `exercici-2`.
2. Dentro de `~/examen` cread un directorio llamado `exercici-2`.
3. Cread `index.html` con este contenido, cambiando el nombre y apellido por el vuestro:

```
<!DOCTYPE html>
<html lang="ca">
<head>
 <meta charset="utf-8">
 <title>Portfoli de Nom Cognom</title>
 <link rel="stylesheet" href="estil.css">
</head>
<body>
 <h1>Portfoli de Nom Cognom</h1>
 <p>Pàgina publicada amb GitHub Pages.</p>
</body>
</html>
```

4. Añadid solo `index.html` y haced un commit con el mensaje `Afegeix pàgina principal`.
5. Añadid un fichero llamado `Readme.md` que contenga el siguiente texto: `# Capçalera`. Añadid solo `Readme.md` y haced un commit con el mensaje `Afegeix readme`.
6. Haced push al repo que habéis creado en GitHub en el paso 1.

Solución

`~/examen` ya es un repo (ejercicio 1) y `exercici-2` va dentro. Entra en `exercici-2` **antes** del `git init` y comprueba con `pwd` dónde estás.

```
# 1. Repo vacío en GitHub
cd ~
gh repo create exercici-2 --public

# 2. Directorio dentro de ~/examen
mkdir ~/examen/exercici-2
cd ~/examen/exercici-2
git init -b main
git remote add origin https://github.com/<usuari>/exercici-2.git

# 3. index.html (cambia Nom Cognom por el tuyo)
cat > index.html << 'EOF'
<!DOCTYPE html>
<html lang="ca">
<head>
 <meta charset="utf-8">
 <title>Portfoli de Nom Cognom</title>
 <link rel="stylesheet" href="estil.css">
</head>
<body>
 <h1>Portfoli de Nom Cognom</h1>
```

```
 <p>Pàgina publicada amb GitHub Pages.</p>
</body>
</html>
EOF

# 4. Solo index.html
git add index.html
git commit -m "Afegeix pàgina principal"

# 5. Readme.md
echo "# Capçalera" > Readme.md
git add Readme.md
git commit -m "Afegeix readme"

# 6. Push
git push -u origin main
```

El `estil.css` no existe y no hace falta crearlo.

Ejercicio 3: .gitignore (1 punto)

Enunciado

1. Acceded a `https://github.com/titansmc/examen-daw`, descargad `exercici-3.zip` y descomprimidlo en vuestro ordenador.
2. Entrad dentro del directorio que habéis descomprimido.
3. Cread un `.gitignore` que ignore el directorio `logs`.
4. Cread un repo en GitHub llamado `exercici-3`.
5. Haced un commit solo con el `.gitignore`, con el mensaje `Afegeix .gitignore`, y subidlo.
6. Haced un segundo commit con el `script.py` y con el mensaje `Afegeix script`, y subidlo.

Solución

Descarga: en el navegador, `github.com/titansmc/examen-daw` → pulsa `exercici-3.zip` → icono de descarga (**Download raw file**). Alternativa: `git clone https://github.com/titansmc/examen-daw` y copiar el zip.

```
cd ~/Descargas                          # o donde se haya descargado
unzip exercici-3.zip
ls                                      # mira cómo se llama el directorio
cd exercici-3
ls -a                                   # comprueba que NO trae ya una carpeta
.git

echo "logs/" > .gitignore

gh repo create exercici-3 --public
git init -b main
git remote add origin https://github.com/<usuari>/exercici-3.git

git add .gitignore
git commit -m "Afegeix .gitignore"
git push -u origin main

git add script.py
git commit -m "Afegeix script"
git push
```

- No uses `git add .`: el enunciado pide commits con archivos concretos.
- `git status` no debe mostrar `logs/` como archivo sin rastrear.
- En GitHub debe haber `.gitignore` y `script.py`, y no `logs/`.

Ejercicio 4: GitHub Pages (2 puntos)

Enunciado

1. Si tenéis un repo con el nombre `<usuari>.github.io`, borradlo y cread uno de nuevo.
2. Id a `https://github.com/titansmc/examen-daw` y descargad `exercici-4.zip`.
3. Subid el contenido al repo que acabáis de crear.
4. Mientras se publica (puede tardar uno o dos minutos), podéis avanzar con el ejercicio 5.
5. En el repositorio de GitHub que acabáis de crear, activad GitHub Pages para que publique la rama `main` desde la raíz (`/ (root)`).
6. Cuando la página esté disponible, editad `README.md` en local y añadid vuestro nombre al final del fichero. Haced commit con el mensaje `Afegeix el meu nom` y subidlo.

Solución

1. Borrar el repo si existe (web: repo → **Settings** → abajo, **Danger Zone** → **Delete this repository**; o por terminal) y **crearlo de nuevo**, vacío y público, con el nombre exacto:

```
gh repo delete <usuari>/<usuari>.github.io --yes
gh repo create <usuari>.github.io --public
```

2-3. Descargar `exercici-4.zip`, descomprimir y subir. El `index.html` debe quedar en la **raíz** del repo; si el zip crea una subcarpeta, entra en ella y sube su contenido.

```
unzip exercici-4.zip
cd exercici-4                       # la carpeta con index.html dentro (usa
ls)
git init -b main
git add .
git commit -m "Primer commit"       # el enunciado no fija este mensaje
git remote add origin https://github.com/<usuari>/<usuari>.github.io.git
git push -u origin main
```

5. Activar Pages (hazlo enseguida y sigue con el ejercicio 5): repo → **Settings** → **Pages** → *Build and deployment* → **Source:** *Deploy from a branch* → **Branch:** `main`, carpeta `/(root)` → **Save**. La web quedará en `https://<usuari>.github.io/`.

6. Cuando la página funcione, añade tu nombre al final de `README.md`:

```
echo "Nom Cognom" >> README.md
git add README.md
git commit -m "Afegeix el meu nom"
git push
```

Si el README no acababa en salto de línea, tu nombre se pegará a la última línea: mejor ábrelo con un editor y ponlo en una línea nueva.

Ejercicio 5: GitHub Actions (3 puntos)

Enunciado

1. Id a `https://github.com/titansmc/examen-daw` y descargad `exercici-5.zip`.
2. Cread un repo en GitHub llamado `exercici-5` y subid a él lo que haréis a continuación.
3. Dentro del directorio veréis dos scripts de Python. Uno es el programa y el otro contiene los *tests*. Vamos a simular mediante GitHub Actions que queremos ejecutar los tests unitarios cada vez que se haga push a la rama `main`.

4. La manera de ejecutar los tests unitarios es mediante la orden: `python -m unittest -v`
5. Tenéis que crear 3 jobs, llamados `test-win`, `test-linux` y `test-mac`, donde se ejecuten los tests unitarios en estos sistemas operativos. Para hacer el checkout del repo y configurar el entorno de Python, podéis hacer uso de las actions siguientes:
   - `actions/checkout@v6`
   - `actions/setup-python@v6`
6. La versión de Python que se quiere utilizar para el job de Windows es la 3.12 y para el de Mac y Linux la 3.13.

Solución

```
unzip exercici-5.zip
cd exercici-5                          # mira el nombre real con ls

gh repo create exercici-5 --public     # repo vacío, sin README
git init -b main
git remote add origin https://github.com/<usuari>/exercici-5.git

mkdir -p .github/workflows
# crear .github/workflows/tests.yml con el contenido de abajo

git add .
git commit -m "Afegeix workflow de tests"
git push -u origin main
```

`.github/workflows/tests.yml` (el nombre del archivo es libre):

```
name: Tests

on:
  push:
    branches: [main]

jobs:
  test-win:
    runs-on: windows-latest
    steps:
      - uses: actions/checkout@v6
      - uses: actions/setup-python@v6
        with:
          python-version: "3.12"
      - run: python -m unittest -v
```

```
  test-linux:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v6
      - uses: actions/setup-python@v6
        with:
          python-version: "3.13"
      - run: python -m unittest -v

  test-mac:
    runs-on: macos-latest
    steps:
      - uses: actions/checkout@v6
      - uses: actions/setup-python@v6
        with:
          python-version: "3.13"
      - run: python -m unittest -v
```

- Los nombres de los jobs deben ser exactamente `test-win`, `test-linux` y `test-mac`.
- Windows = `windows-latest` con 3.12. Linux = `ubuntu-latest` y Mac = `macos-latest`, con 3.13.
- `checkout` va primero: sin él el runner está vacío y `unittest` no encuentra los tests.
- Subir un workflow necesita el permiso `workflow` del token (por eso se marcan todos los permisos).
- Comprobación: pestaña **Actions** del repo → una ejecución con tres jobs en verde. Abre un job y el step `Run python -m unittest -v` para ver los tests.

Resumen de mensajes de commit exactos

Ejercicio	Mensajes
1	`Primer commit`, `Segon commit`, `Tercer commit`
2	`Afegeix pàgina principal`, `Afegeix readme`
3	`Afegeix .gitignore`, `Afegeix script`
4	`Afegeix el meu nom` (el primer commit no está fijado)
5	Libre; la pestaña Actions debe mostrar los 3 jobs en verde